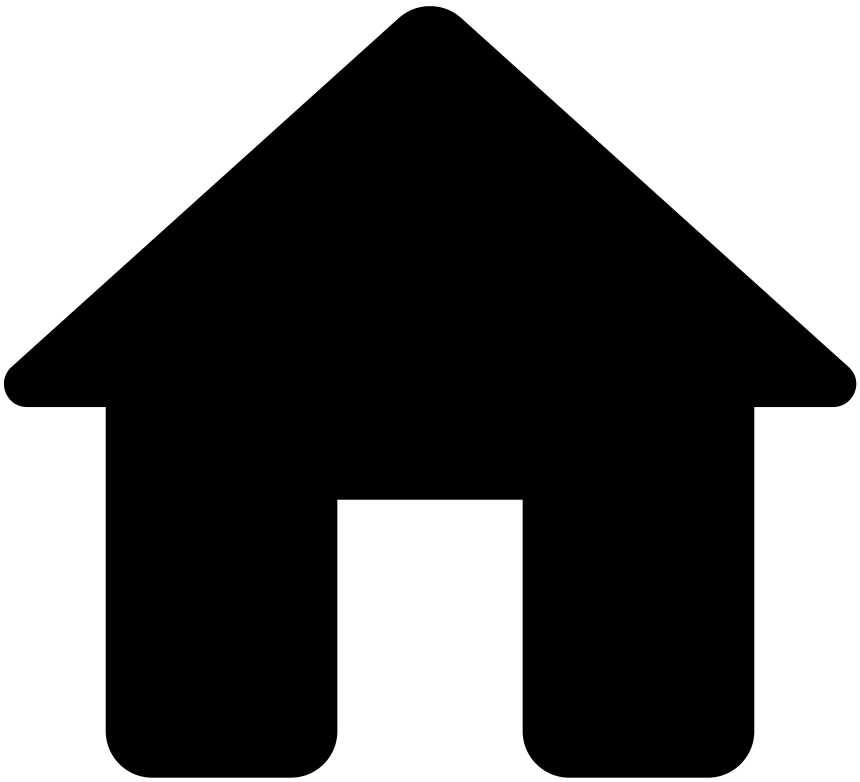


Logging

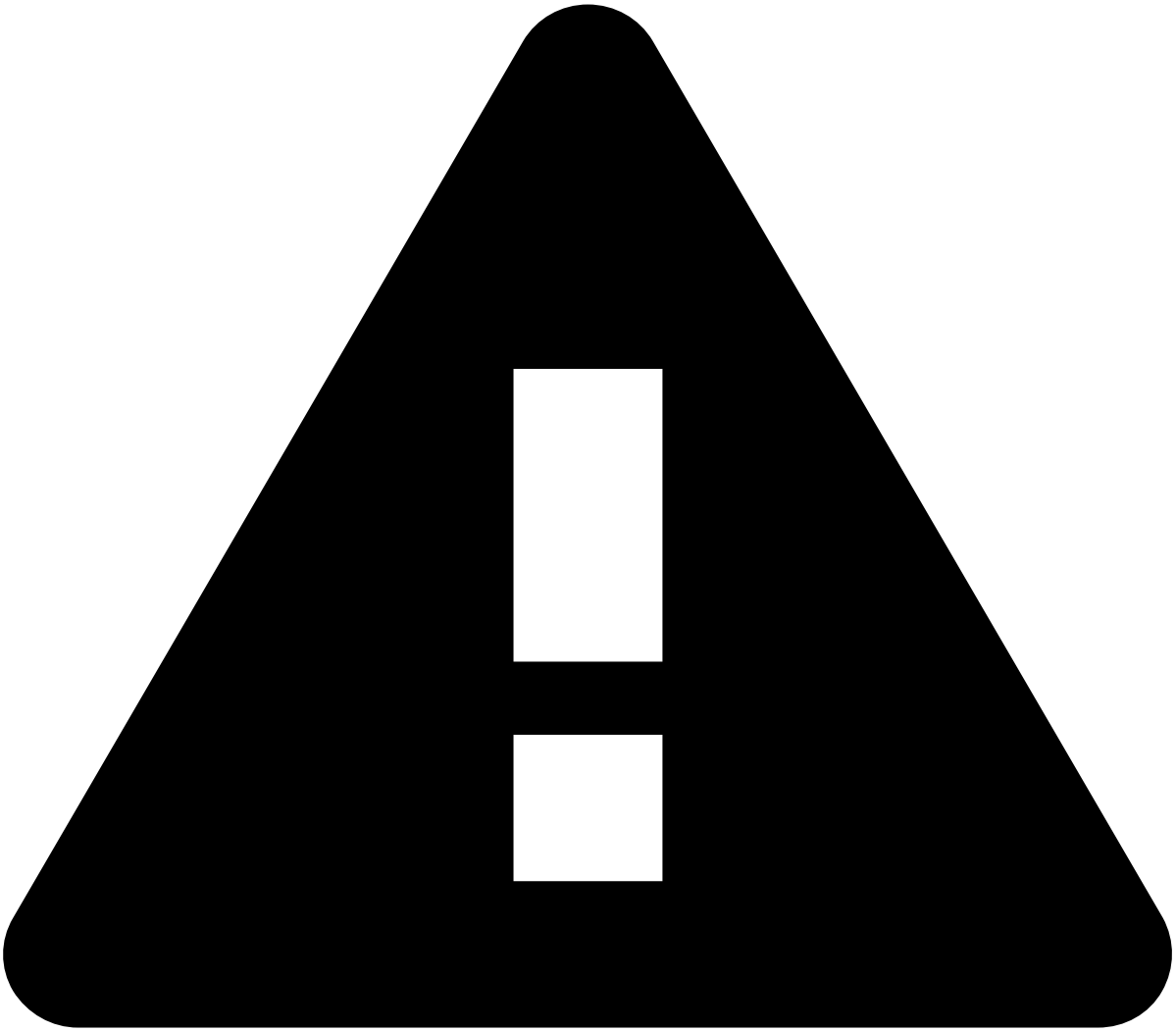
All operations executed internally by Pulse are logged according to several levels of detail in log files located in the `PULSE_HOME/log` directory.

Pulse uses SLF4J logging facade making all the log calls agnostic to the actual logging framework in place. This allows changing the logging framework at deployment time if necessary. Nevertheless, Pulse is distributing Logback logging framework by default and this page discusses how to configure it.

Configuringâ

The logging configuration file is located in `PULSE_HOME/conf/logger.xml`. This file includes different configuration parts from files present in `PULSE_HOME/conf/logger/*`.

For a more advanced configuration not covered in this article please consult the Logback official documentation.



warning

Changing the logging configuration is on one's own responsibility. Changes might result in the loss of valuable information for investigating problems. If the default configuration is changed, support teams might not be able to help so one should be extremely careful when editing this file.

Logging configuration filesâ

As mentioned above, the main logging configuration file is located in **PULSE_HOME/conf/logger.xml**. This file shouldn't be changed by clients as it does the inclusion of other logging configuration files present in **PULSE_HOME/conf/logger/***:

- **jmxconfigurator.xml**: Enables exporting logger configurations via JMX.
- **filters.xml**: Defines filters to be applied to log messages.
- **conversion-rules.xml**: Formatting rules for log messages.
- **default-appenders.xml**: Defines the default appenders enabled in Pulse. **Clients in production should change this file** to disable BUFFER and STDOUT appenders.

- **logger-jupyterlab.xml**: Logger and appender configuration to send all JupyterLab logs to its own file - *log/jupyterlab.log*
- **logger-confidential.xml**: Logger and appender configuration to send all confidential logs to its own file - *log/CONFIDENTIAL-*.log*
- **logger-metrics.xml**: Logger and appender configuration to log probes to disk.
- **logger-legacy-audit.xml**: Logger and appender configuration to log legacy audit messages to disk.
- **logger-audit.xml**: Logger and appender configuration to log new audit entries to disk.
- **logger-events.xml**: Logger and appender configuration to log events to disk.
- **logger-rules.xml**: Logger and appender configuration to log rules to disk.
- **logger-default.xml**: Default logger configurations that are shipped with Pulse.
- **logger-project.xml**: File that needs to be created by clients if they need to override logger definitions that are shipped with Pulse or if they need to add new logging configurations such as appenders or loggers.

Log Filesâ□□

In order to easily pin point possible issues when running Pulse, Pulse Server and Applications write their messages to different log files.

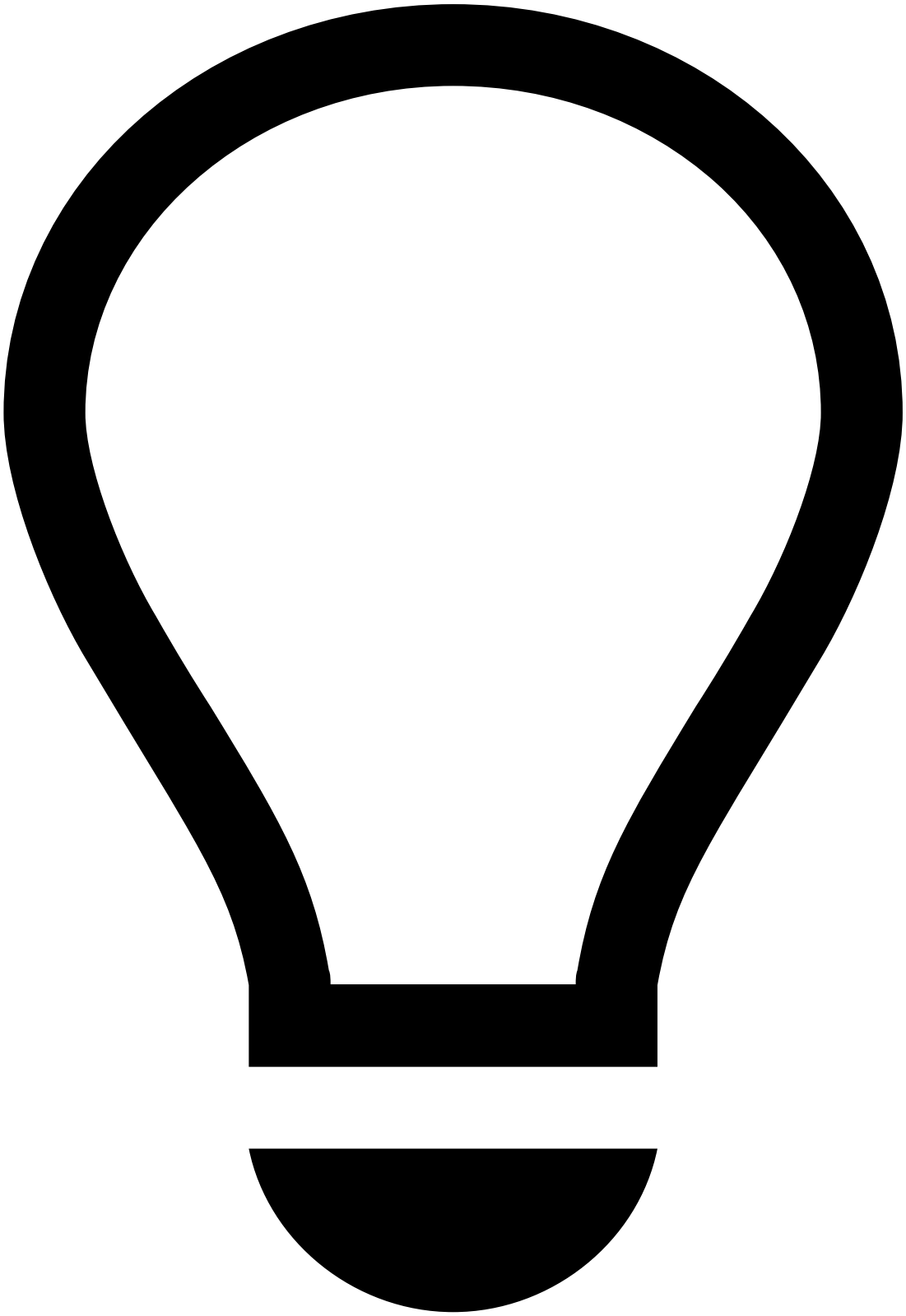
Whatever the configuration in place might be, Pulse will always generate the following:

- **pulse.log**: Logs all operations performed inside Pulse server
- **app-<APP NAME>.log**: Logs all operations performed in an application engine

For additional GC (Garbage Collection) logs, the following flags were added by default to the JVM_OPTS of the Pulse and the applications:

- **-Xloggc:log/pulse-%pid.gc -XX:+PrintGCDateStamps -XX:+PrintGCApplicationStoppedTime**

They will appear in a format similar to **<pulse|APP_SLUG>_GC_pid<PID>_<DATE>_.log.0.current** inside the logging folder.



tip

If the JVM is already running and it did not happen to be started with the necessary JVM_OPTS, it is still possible to enable some of them dynamically using the jinfo tool, e.g.:

- **jinfo -flag +PrintGCDateStamps <pid>**
- **jinfo -flag +PrintGC <pid>**

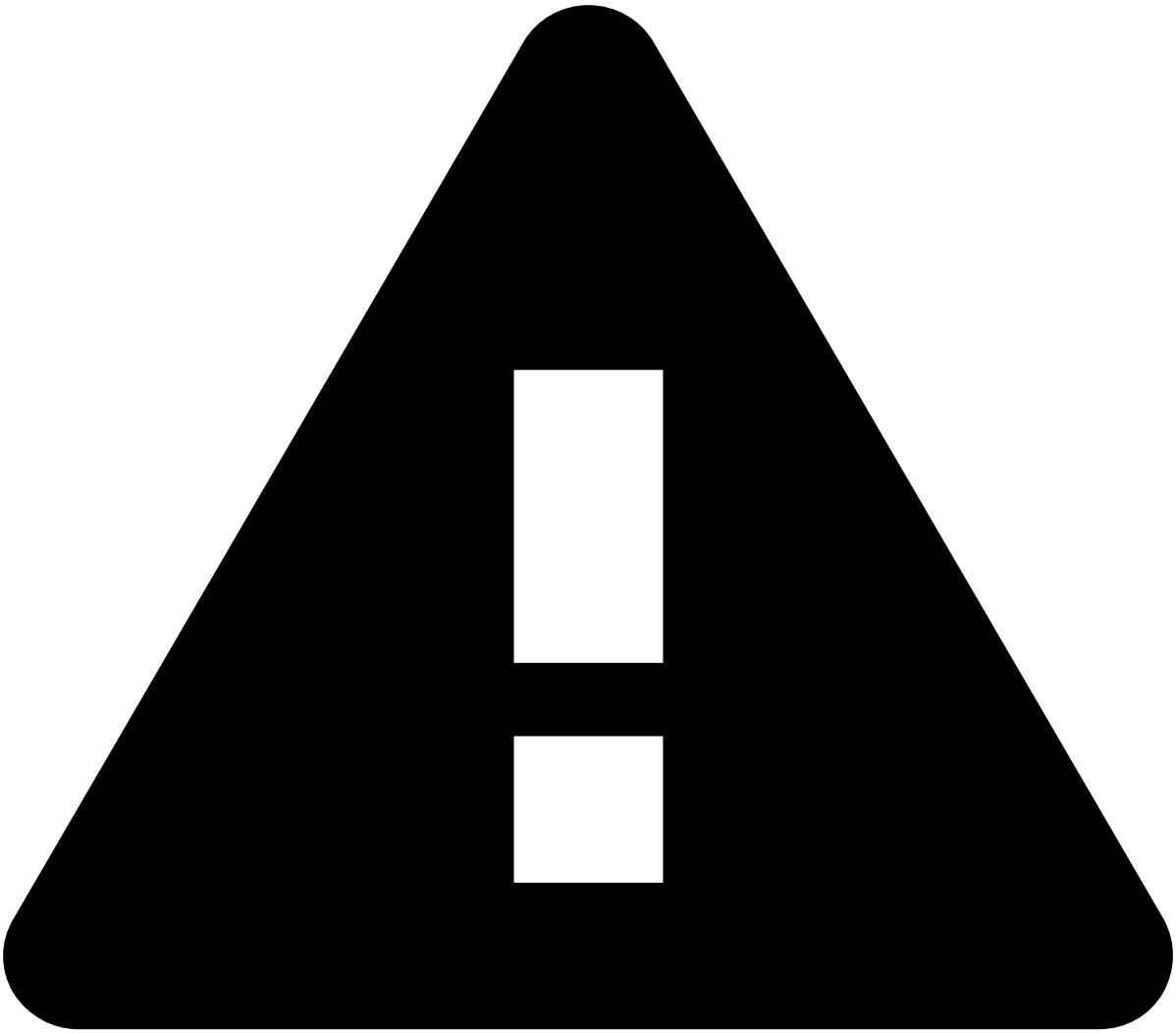
DEV Markerâ□□

The development marker provides a much refined logging useful when developing Pulse Services and web applications. To turn it **on** or **off** just set the **OnMatch** tag to **ACCEPT** or **DENY** accordingly.

Appenders Sectionâ□□

The appenders section define where the logging messages will end up. By default, Pulse is configured with three appenders: **STDOUT**, **FILE** and **BUFFER**.

- **STDOUT**: Defines how messages are presented in the standard output (console);
- **FILE**: Defines how messages are presented in the log file (for both server and applications). By default these files are rolled every week (each week will be in a zipped file) and a maximum of 8 week worth of history is kept;
- **BUFFER**: Defines how messages are presented in the UI.



Do not use stdout append in PROD

It is *not* recommended to have the STDOUT appended in production scenarios as it may cause the app-engines to block in some extreme error scenarios (for example REQ-2666).

An appender is defined in the configuration file with the `<appender>` XML tag. Inside each appender tag, there are several options mainly the **filter** and the **encoder**.

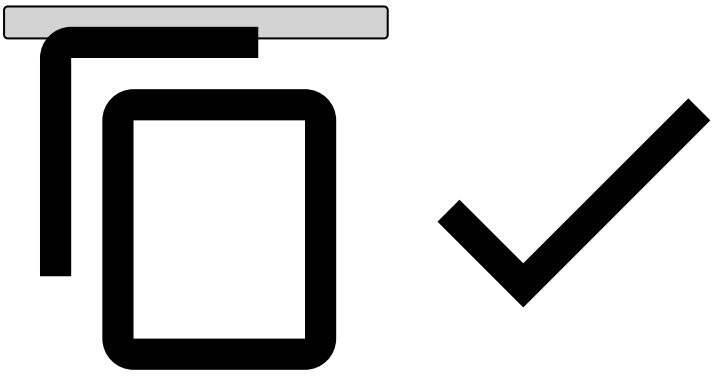
- **FILTER**: Defines the minimum logging necessary (inclusively) in order for the encoder to be used;

- **ENCODER**: Defines tags and colors for the logging messages given the package of the logger in use.

Under **filter** all available logging levels can be used (**ERROR**, **WARN**, **INFO**, **DEBUG** and **TRACE**). When you define a logging level, that level and the above ones (more critical than the one defined) will always satisfy the filter. For instance if you define **INFO**, then the **WARN** and **ERROR** messages are also logged.

With the encoder it's possible to define the timestamp format of the logging events and tag them with colors and labels. The following is an example of tagging the all the messages that have to do with the rules engine.

```
<encoder class="ch.qos.logback.core.encoder.LayoutWrappingEncoder">
    <layout class="com.feedzai.pulse.server.manager.logging.LoggerLayout">
        <colored>true</colored>
        <!-- Abbreviate the package -->
        <abbrevLevel>true</abbrevLevel>
        <!-- Show stacktrace -->
        <showTraces>false</showTraces>
        <!-- Color of the levels -->
        <debugColor>1;30</debugColor>
        <infoColor>0</infoColor>
        <stdioColor>0;35</stdioColor>
        <tagColor>1;30</tagColor>
        <!-- Timestamp format -->
        <timestampFormat>dd-MMM HH:mm:ss</timestampFormat>

        <!-- Tag the events the match the package with text -->
        <tagEvents>
            <from>com.feedzai.pulse.service.rules</from>
            <with>RULES</with>
            <color>1|1;33</color>
        </tagEvents>
    </layout>
</encoder>
```

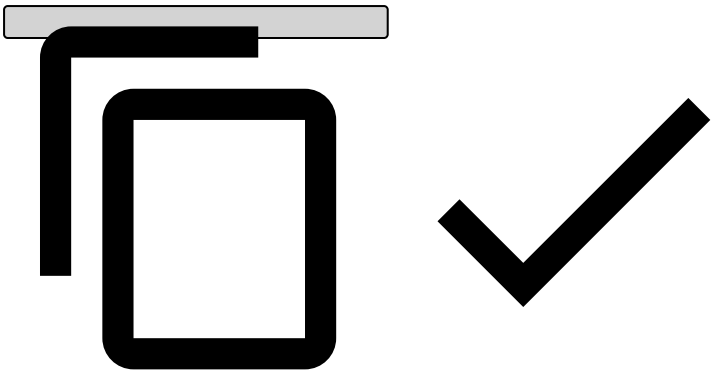


Logger Sectionâ□□

The logger section is where the main level filtering is made. Here levels are assigned to packages or even to their classes using the **name** property. This configuration can be overridden ultimately in the final appenders. For instance if you define here that a logger is enabled at the **DEBUG** level, but an appender is specified to only allow above **INFO**, the messages won't appear in that appender.

It's also possible to assign a logger some configuration. For instance you might want for all the logging messages of your application to go inside a specific file. For that you could create an appender and forward all the messages to go there.

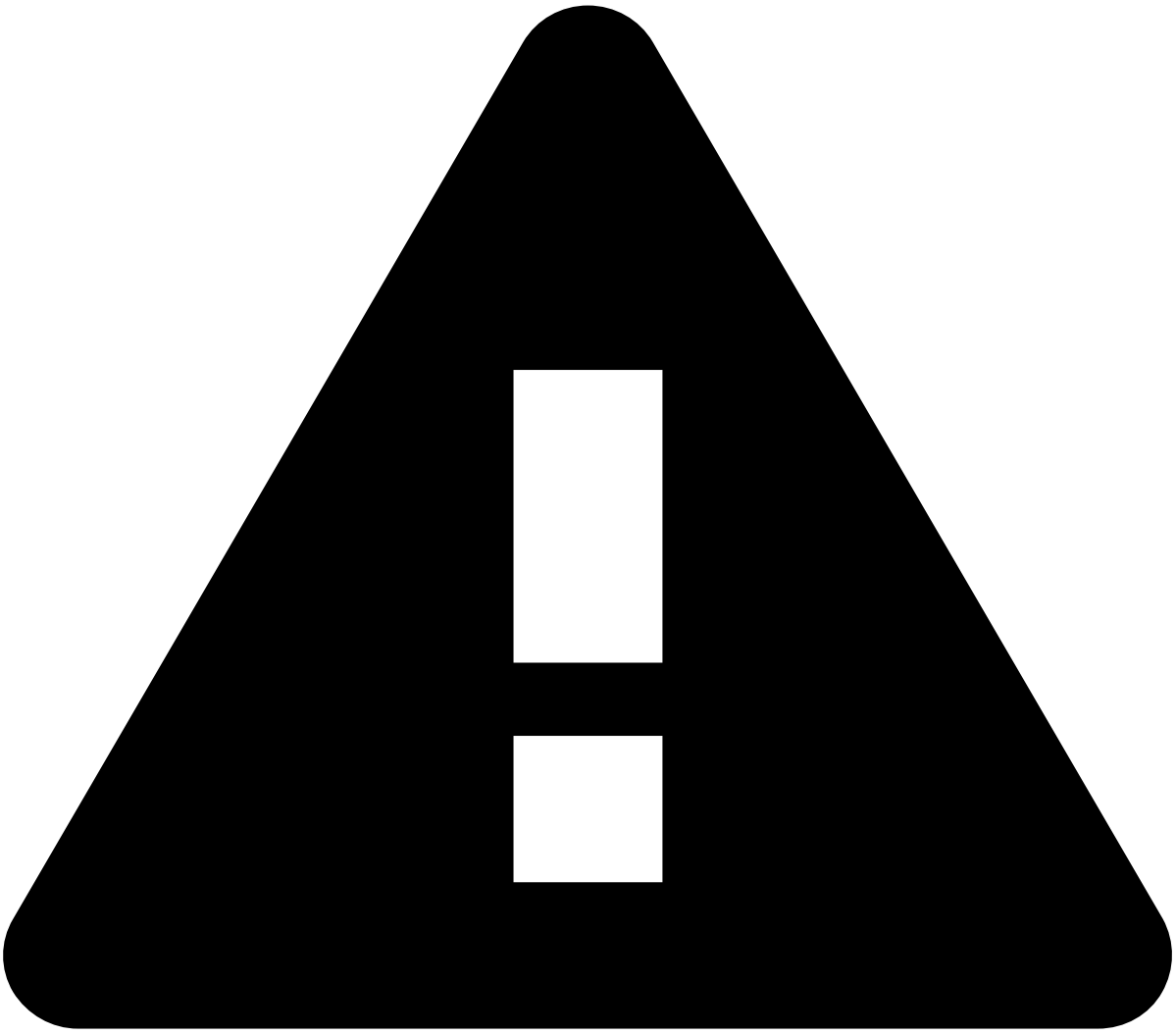
```
 <appender name="MY_SERVICE" class="ch.qos.logback.core.rolling.RollingFileAppender">
    <file>log/my_service.log</file>
</appender>

<logger name="com.company.my.service" additivity="false">
    <appender-ref ref="MY_SERVICE" />
</logger>
```



The **additivity** parameter tells the logger that it mustn't be used in other loggers and therefore appenders. In this case, logging calls to that logger will not reach the **CONSOLE**, **BUFFER** or **FILE** appenders but only the **MY_SERVICE** one.

Root Loggerâ□□

The root logger is the default logger where all the messages will wind up by default. The only case where that doesn't happen is when a logger is defined with the **additivity** parameter as **false**. Here are usually defined the destination appenders of the messages.



caution

We strongly advise not to change the appenders already under the root logger section.

Lnav Supportâ□□

Lnav is a log viewer that can help read and analyze logs. Among other features, it offers highlighting and a unified view over multiple log files.

For Lnav to support Pulse's log format you need to install the Pulse log lnav configuration:

```
lnav -i feedzai-pulse-log-format.json
```

This log format is valid for Pulse 15.1.5 and above. For previous 15.1 versions you should use this configuration file.

To launch lnav, do:

```
lnav <pulse-path>/log/
```